

2.6 - A complexidade real dos microsserviços

O preço que ninguém coloca no slide de vendas

A nova classe de problemas

O que o monólito nunca enfrentou

- **A rede entra como nova fonte de falha**
 - Latência e timeout viram preocupação constante
- **Uma requisição atravessa vários serviços**
 - Rastrear a origem de um erro fica difícil
- **Deploys interdependentes exigem coordenação**
 - *A ordem de subida dos serviços passa a importar*
- **Dezenas de repositórios, pipelines e configurações**
 - O overhead operacional cresce com cada serviço novo

O monólito não tinha vs. microserviços exigem

O monólito não precisava de

- Tratamento de falha de rede interna
- Rastreamento distribuído de requisições
- Service discovery e roteamento dinâmico
- Versionamento de contratos entre serviços

Microserviços exigem

- Timeout, retry e circuit breaker explícitos
- Tracing com correlação de IDs
- API gateway e descoberta de serviços
- Pipeline e observabilidade por serviço

Quatro serviços, quatro novas responsabilidades

- **Com quatro serviços em produção, faltava infraestrutura**
 - Nada disso existia no tempo do monólito

LUNALOG

Com quatro serviços em produção, a LunaLog percebeu que precisava de service discovery, logging centralizado, distributed tracing e API gateway. Nada disso existia no tempo do monólito. O que parecia simplificar o deploy adicionou quatro novas responsabilidades operacionais ao time. Não foi um erro de decisão, foi a realidade que ninguém tinha calculado. Roberto resumiu numa frase: a gente trocou um problema grande por vários problemas médios.

O overhead operacional invisível

- **Cada serviço precisa de pipeline e deploy próprios**
 - Multiplicar serviços multiplica o trabalho de plataforma
- **Observabilidade deixa de ser opcional**
 - Sem tracing, depurar produção vira adivinhação
- **Falha parcial passa a ser o estado normal**
 - *Parte do sistema pode estar fora a qualquer momento*
- **O custo cognitivo se espalha por todo o time**
 - Manter o sistema inteiro na cabeça fica impossível

Microserviços trocam complexidade, não eliminam

- **A complexidade não some, ela muda de lugar**
- **Você ganha autonomia e paga em operação**

Microserviços movem a complexidade do código para a operação. Só vale a pena quando autonomia de times e escala independente compensam esse custo. Caso contrário, é dor sem retorno.

- Tenha plataforma e observabilidade antes de dividir



Microserviços têm complexidade real, isso ficou claro. Mas e se, para certos casos, você pudesse ter os benefícios da distribuição sem gerenciar um único servidor? Sem provisionar, sem escalar, sem pagar pela ociosidade? Parece bom demais, e como toda abstração poderosa, tem um preço escondido.

Serverless: modelo de execução e casos de uso